Computational Tools for Modelling and Analysis: Fitting Models to Data

Dr Iain Styles

October 19, 2015

1 *Introduction*

Scientists often try to understand their experimental results in terms of a mathematical model of the experiment. Such models typically have some "free parameters" that need to be found for each experiment, and which tell us something about the behaviour and properties of the experimental system. For example, if we are measuring the position of some cells or particles moving along a narrow linear channel in a time-lapse experiment, we could model their motion by $s = ut + \frac{1}{2}at^2$, where the unknowns are u, the initial velocity and a, the acceleration. The question we address here is how to calculate these free parameters to maximise the model–data correspondence. We will consider three particular cases:

- Linear models: $y = mx + c$, $s = ut + \frac{1}{2}at^2$ (note: this is linear in the *unknowns*).
- Linear models with measurement error.
- Nonlinear models: $y = a + b\sin(cx)$, $y = axe^{-a^2x}$

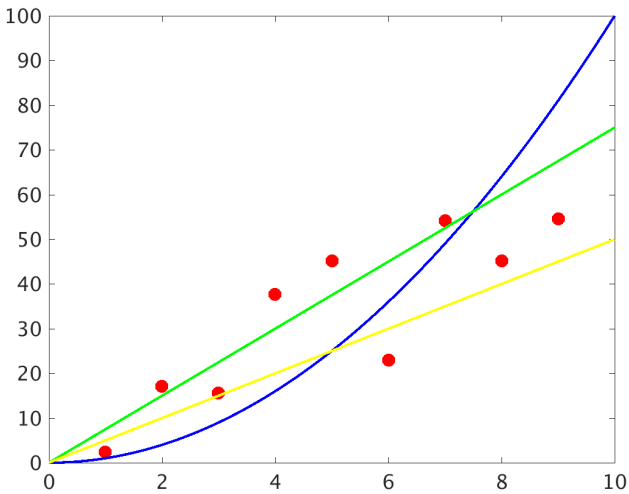

Figure 1: A set of data points together with three possible lines can could be used to model them. These notes describe how to find the optimal line to model the points.

2 *Least squares Fitting*

A fundamental concept in model fitting that the "least-squares error", which provides a way of quantifying the difference between a set of data points and a mathematical model. Imagine we have a set of data points $\mathcal{D}$ consisting of n (x, y) pairs, where x is some experimental condition (e.g. time, concentration, voltage) and y is

the measured quantity:

$$\mathcal{D} = \{(x_1, y_1), \ldots, (x_n, y_n)\} \tag{1}$$

Let our modelling function be defined as

$$y = f(\mathbf{p}, x) \tag{2}$$

where $\mathbf{p} = (p_1, \ldots, p_m)$ is a vector of unknown parameters[1]. At each value of x, the difference between the model f and the measurements is a function of $\mathbf{p}$ and is called the **residual**:

[1] For example, a straight line model $y = mx + c$ has $\mathbf{p} = (m, c)$.

$$r_i(\mathbf{p}) = y_i - f(\mathbf{p}, x_i). \tag{3}$$

The *least squares error* is then defined as

$$e(\mathbf{p}) = \sum_{i=1}^{n} r_i^2 = \mathbf{r}^{\mathrm{T}}\mathbf{r} \tag{4}$$

and our fitting problem amounts to finding the value of $\mathbf{p} = \mathbf{p}^*$ that minimised the error:

$$\mathbf{p}^* = \arg\min_{\mathbf{p}} e(\mathbf{p}) \tag{5}$$

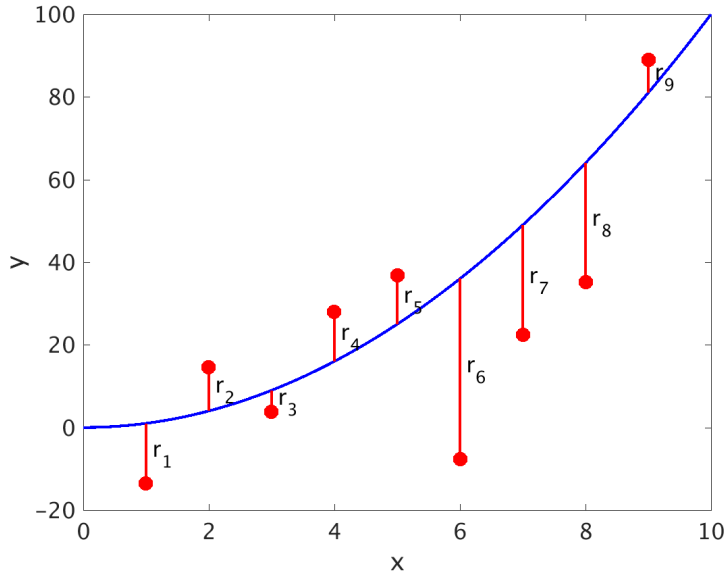

Figure 2: The residuals (as marked on the diagram) are the differences between a particular model and the set of data points.

3 *Linear Models*

The simplest case is that of models that are linear in the unknown parameters. Straight lines, $f(\mathbf{p}, x) = p_1 x + p_2$ are a linear model, but in general we consider models of the form

$$f(\mathbf{p}, x) = p_1 f_1(x) + \cdots + p_m f_m(x) = \sum_{j=1}^{m} p_j f_j(x) \tag{6}$$

and the residuals are therefore

$$r(\mathbf{p}, x_i) = y_i - \sum_{j=1}^{m} p_j f_j(x_i) \tag{7}$$

The f_i can be any function of x, but there cannot be any unknowns therein. Treating $\mathbf{p}$ as a column vector and defining the component of $\mathbf{F}$ as $F_{ij} = f_j(x_i)$ we can re-write this as the matrix equation

$$\mathbf{r} = \mathbf{y} - \mathbf{F}\mathbf{p} \tag{8}$$

where $\mathbf{y} = (y_1, \ldots, y_n)$. As an example consider a simple quadratic model $f(\mathbf{p}, x) = p_1 x^2 + p_2 x + p_3$. Each row of $\mathbf{F}$ corresponds to a data point, whilst each column corresponds to a term in the model:

$$\begin{pmatrix} r_1 \\ r_2 \\ \vdots \\ r_n \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} - \begin{pmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ & \vdots & \\ x_n^2 & x_n & 1 \end{pmatrix} \begin{pmatrix} p_1 \\ p_2 \\ p_3 \end{pmatrix}. \tag{9}$$

In Matlab, we can form the matrix F in this case as

```
F = [x.^2, x, x.^0];
```

Our least-squares error can therefore be written

$$e(\mathbf{p}) = (\mathbf{y} - \mathbf{F}\mathbf{p})^{\mathrm{T}} (\mathbf{y} - \mathbf{F}\mathbf{p}) \tag{10}$$

4 *Finding the Best Fit*

The solution to our problem of finding the parameters of "best fit" requires us to find the value of $\mathbf{p} = \mathbf{p}^*$ that minimised $e(\mathbf{p})$. To find the solution, we find the minimum by differentiating with respect to $\mathbf{p}$ and setting the result to zero:

$$\mathbf{F}^{\mathrm{T}}\mathbf{y} - \mathbf{F}^{\mathrm{T}}\mathbf{F}\mathbf{p}^* = 0, \tag{11}$$

a set of simultaneous linear equation that we could either solve by direct inversion:

$$\mathbf{p}^* = \left(\mathbf{F}^{\mathrm{T}}\mathbf{F}\right)^{-1} \mathbf{F}^{\mathrm{T}}\mathbf{y}; \tag{12}$$

or in Matlab using Gaussian elimination through the \ operator:

```
pstar = (F'*F)\(F'*y)
```

At this point you may be tempted to ask the following question: we have modelled our data as $\mathbf{y} = \mathbf{F}\mathbf{p}$, so why not just solve this directly? There are two reasons:

- The calculations we have done guarantee the specific solution we will obtain (the least square solution) and we are not at the whim of the solver.
- In the next section we will want to incorporate some extra information into the calculation which we must add directly into $e(\mathbf{p})$.

5 *Weighted Linear Least Squares*

Our analysis so far has assumed that for every value of x, we have one value of y, and we want to fit a model to those single points. This will frequently not be the case as many experiments will be repeated multiple times. Clearly we can straightforwardly fit models to the "average" of a set of points (be that the mean or the median), but is this the right thing to do? The particular issue we are concerned with is how to account for the spread (variance) in the data in the fitting process.

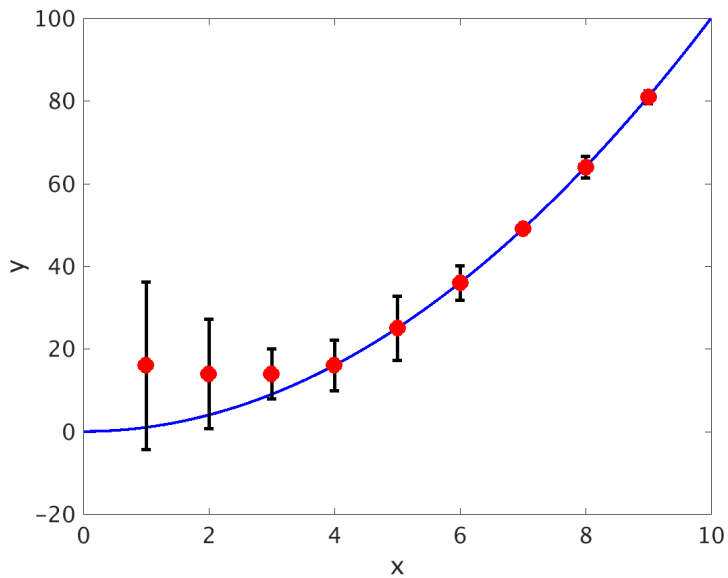

Figure 3: A set of data points with error bars and a line of best fit. This line could be chosen by treating all points equally, or by preferentially weighting those that are known most precisely.

If a data point has a small variance, then we can usually assume that its value has been determined quite precisely by the measurement, and therefore we would want any model to fit that point accurately. If the variance is large though, we know the value much less precisely, and perhaps a less precise fit is necessary at that point. To include this consideration in the fitting procedure, we need to modify the way in which we calculate the error. So far, we have "weighted" all residuals equally in $e(\mathbf{p})$, but it is straightforward to add an additional weighting term. Naïvely we would write

$$e(\mathbf{p}) = \sum_{i=1}^{n} w_i r_i^2 \tag{13}$$

where the w_i weight contributions from the different residuals. We will define $w_i = 1/\operatorname{var}(y_i)$ such that terms with a small variance are more heavily weighted in the error function than those with a large variance.

It will turn out to be mathematically more convenient if we de-

fine the following matrix:

$$\mathbf{\Sigma} = \begin{pmatrix} \mathrm{var}(y_1) & 0\ldots & \ldots & 0 \\ 0 & \mathrm{var}(y_2) & 0\ldots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & \ldots & 0 & \mathrm{var}(y_n) \end{pmatrix}. \tag{14}$$

[2]

[2] This is a special case of the **covariance matrix** that we will meet again in later parts of the course.

This allows us to write the error as

$$e(\mathbf{p}) = \mathbf{r}^{\mathrm{T}}\mathbf{\Sigma}^{-1}\mathbf{r} \tag{15}$$

$$= (\mathbf{y} - \mathbf{F}\mathbf{p})^{\mathrm{T}}\,\mathbf{\Sigma}^{-1}\,(\mathbf{y} - \mathbf{F}\mathbf{p}) \tag{16}$$

where we note that $\mathbf{\Sigma}^{-1}$ requires only the diagonal elements to be inverted to ensure $\mathbf{\Sigma}^{-1}\mathbf{\Sigma} = \mathbf{I}$.

We perform a similar analysis (differentiate and set to zero to find the minimum in the error) as in the unweighted case to obtain the final result

$$\mathbf{F}^{\mathrm{T}}\mathbf{\Sigma}^{-1}\mathbf{y} - \mathbf{F}^{\mathrm{T}}\mathbf{\Sigma}^{-1}\mathbf{F}\mathbf{p}^{*} = 0, \tag{17}$$

which can be solved using the methods we have already established.

6 *Fitting Nonlinear Models*

The analysis to date is restricted only to models that ca be expressed in the form given in Eq. (6), but there may be many cases where this is not possible (eg. $y(x) = a \sin bx$). We still have a desire to minimise the sum of the squares of the residuals, but we cannot use the linearity of the fitting function to enable a single-step solution. We must formulate a process that instead of solving Eq. (5) to find the value of $\mathbf{p}$ that minimises the error, finds a set of parameters that simply reduces the error. We will do this iteratively until no further reduction in error is possible.

We will proceed by examining the behaviour of the error when we change $\mathbf{p}$ by a small amount. This we do by approximating the function as a straight line over a small region using *Taylor's Theorem*:

$$f(x_i, \mathbf{p} + \mathbf{\Delta p}) \approx f(x_i, \mathbf{p}) + \sum_{j=1}^{m} \frac{\partial f(x_i)}{\partial p_j} \Delta p_j \tag{18}$$

$$= f(x_i, \mathbf{p}) + \sum_{j=1}^{m} J_{ij}\delta p_j \tag{19}$$

and the residual after the change is

$$\mathbf{r}(\mathbf{p} + \mathbf{\Delta p}) = \mathbf{y} - \mathbf{f}(\mathbf{p}) - \mathbf{J}\mathbf{\Delta p} \tag{20}$$

where $f_i(\mathbf{p}) = f(x_i, \mathbf{p})$. The error is therefore

$$e(\mathbf{p} + \mathbf{\Delta p}) = (\mathbf{y} - \mathbf{f}(\mathbf{p}) - \mathbf{J}\mathbf{\Delta p})^{\mathrm{T}}\,(\mathbf{y} - \mathbf{f}(\mathbf{p}) - \mathbf{J}\mathbf{\Delta p})\,. \tag{21}$$

By minimising this quantity, we find the *change* in $\mathbf{p}$ that gives us the largest reduction in the error. We differentiate as before and set to zero:

$$\left(\mathbf{J}^{\mathrm{T}}\mathbf{J}\right)\Delta\mathbf{p}^{*} = \mathbf{J}^{\mathrm{T}}\left(\mathbf{y} - \mathbf{f}(\mathbf{p})\right) \tag{22}$$

which we know how to solve.

It is common practice to make a small modification to this:

$$\left(\mathbf{J}^{\mathrm{T}}\mathbf{J} + \lambda\,\mathrm{diag}(\mathbf{J}^{\mathrm{T}}\mathbf{J})\right)\Delta\mathbf{p}^{*} = \mathbf{J}^{\mathrm{T}}\left(\mathbf{y} - \mathbf{f}(\mathbf{p})\right) \tag{23}$$

Without this modification, the parameter vector will change quickly along directions with steep gradients, but only slowly along shallow gradients. The additional term increases movement along shallow gradients and thus leads to a faster convergence. The amount of increase is controlled by the damping factor λ which is modified iteratively to give the best results. The full algorithm, named after its inventors Levenberg and Marquadt is described in Algorithm 1.

7 Some New Matlab Constructions

7.1 Concatenating Matrices

In an earlier example, we formed

```
F = [x.^2, x, x.^0];
```

As `x` is a column vector, and the operators are element-wise, Matlab knows to expand this for each row of `x`, and so this provides a nice shorthand for forming the matrices for the fitting problems we have studied in this section. We could also have written:

```
F = []; % Create an empty matrix
F = [x.^0, F] % Concatenate x.^0 with the empty matrix
F = [x.^1, F] % Concatenate x with F
F = [x.^2, F] % Concatenate x.^2 with F
```

7.2 Some other useful matrix functions

```
y = reshape(x,nr,nc) % reshape x to be nr * nc, result to y
[nr,nc] = size(x) % Get the size of x: nr(ows), nc(olumns)
x = ones(nr,nc) % Create a matrix of all ones with nr rows and nc columns
x = zeros(nr,nc) % Create a matrix of all zeros with nr rows and nc columns
X = diag(x) % Given vector x create matrix X with x on its diagonal
x = diag(X) % Given matrix X extract its diagonal into vector x
```

7.3 The `for` loop

Used to repeat a set of commands a fixed number of times.

```
for i=1:N
  % Do stuff
end
```

For example, add up the numbers from 1 to 10:

```
x=0;
for i=1:10
  x=x+i
end
```

7.4 *The `while` loop*

Used to repeat a set of commands until some condition is met.

```
while(condition_is_true)
  % Do stuff
end
```

For example, add up the numbers from 1 to 10 (you wouldn't normally use this type of loop for this):

```
i=0;
x=0;
while(i<=10)
  i=i+1
  x=x+i
end
```

7.5 *`if-else-elseif` Statements*

These are used to perform different actions under different conditions:

```
if(condition A is true)
  % Do X
elseif(condition B is true) % ONLY IF CONDITION A IS FALSE
  % Do Y
else % ONLY IF CONDITION A AND CONDITION B ARE BOTH FALSE
  % Do Z
end
```

There can be one and only one `if`, one and only one `end`, up to one `else` and as many `elseif` as required.

7.6 *Functions*

We have already used some simple mathematical functions such as `sin`, and it is possible to create our own. In Matlab, functions must be in separate files that must be named after them. For example, the following function must be placed on its own in a separate file called `createFMatrix.m`:

```
[F] = function createFMatrix(x)
   F = [x.^2, x, x.^0];
end
```

and we can then call `F = createFMatrix(x)` in the main code.

This provide a mechanism for creating **reuseable code**.

If we only want to create and use a fairly simple function within our current script, we can create a so-called *anonymous function*:

```
createFMatrix = @(x) [x.^2, x, x.^0];
```

In Matlab, function are, in the computer science jargon, *first class citizens*, and so they can be passed around as arguments to other functions. This is extremely useful, and you will see an example of this in the exercises.

Data: Data $\mathbf{x}, \mathbf{y}$; Initial parameter guess $\mathbf{p}_0$; Function $f(x, \mathbf{p})$;
Jacobian $Jac(x, \mathbf{p})$; initial values of λ, ν.
Result: $\mathbf{p}, e(\mathbf{p})$, parameters that minimmise the least squares error, the error.

```
% Evaluate the error at the initial parameter value
```
$\mathbf{p} \leftarrow \mathbf{p}_0$;
$e \leftarrow (\mathbf{y} - f(x, \mathbf{p})^{\mathrm{T}}(\mathbf{y} - f(x, \mathbf{p})$;
```
% Iterate until convergence
```
Converged $\leftarrow$ *False*;
while *Converged* == *False* **do**
```
% Calculate the Jacobian
```
$J \leftarrow Jac(x, \mathbf{p})$;
```
% Calculate update for different values of λ until
  we improve the error
```
LambdaSet $\leftarrow$ *false*;
while *LambdaSet*==*false* **do**
```
% Calculate Δp for λ = λ0 and λ = λ0/ν
```
$\Delta_1 = \left(\mathbf{J}^{\mathrm{T}}\mathbf{J} + \lambda \operatorname{diag}(\mathbf{J}^{\mathrm{T}}\mathbf{J})\right) \setminus \mathbf{J}^{\mathrm{T}}\left(\mathbf{y} - \mathbf{f}(\mathbf{p})\right)$;
$\Delta_2 = \left(\mathbf{J}^{\mathrm{T}}\mathbf{J} + (\lambda/\nu) \operatorname{diag}(\mathbf{J}^{\mathrm{T}}\mathbf{J})\right) \setminus \mathbf{J}^{\mathrm{T}}\left(\mathbf{y} - \mathbf{f}(\mathbf{p})\right)$;
```
% Calculate the error at the new parameter values
```
$e_1 \leftarrow (\mathbf{y} - f(x, \mathbf{p} + \Delta_1)^{\mathrm{T}}(\mathbf{y} - f(x, \mathbf{p})$;
$e_2 \leftarrow (\mathbf{y} - f(x, \mathbf{p} + \Delta_2)^{\mathrm{T}}(\mathbf{y} - f(x, \mathbf{p})$;
if $e_1 < e$ **then**
```
% Set new values of p and e, storing old value
  as e′
```
$\mathbf{p} \leftarrow \mathbf{p} + \Delta_1$;
$e' \leftarrow e$;
$e \leftarrow e_1$;
```
% Current λ is OK
```
LambdaSet $\leftarrow$ *true*;
else if $e_2 < e$ **then**
```
% Set new values of p and e, storing old value
  as e′
```
$\mathbf{p} \leftarrow \mathbf{p} + \Delta_2$;
$e' \leftarrow e$;
$e \leftarrow e_2$;
$\lambda \leftarrow \lambda/\nu$;
```
% Current λ is OK
```
LambdaSet $\leftarrow$ *true*;
else
$\lambda \leftarrow \lambda/ \times nu$;
end
if $e'/e <$ *TerminationCriterion* **then**
Converged $\leftarrow$ *true*
end
end

Algorithm 1: The Levenberg-Marquadt Algorithm.